Analytical Queueing Network Modeling

Table of Contents

Analytical Queueing Network Modeling 1
Table of Figures 1
Introduction 2
Instrumentation and Execution 3
Collected Metrics 3
Traces and metrics processing 4
Modeling and Analysis 4
Determining service rates 6
Building the routing table 7
Solving Traffic Equations 7
Comparison with empirical results 8

Table of Figures

Figure 1: System Architecture 2
Figure 1: system architecture 2
Figure 2: Home QN 4
Figure 3: Product QN 4
Figure 4: ViewCart QN 4
Figure 5: PlaceOrder QN 5
Figure 6: AddToCart QN, EmptyCart QN, Logout QN and SetCurrency QN 5
Figure 7: Service rate - traces 6
Figure 8: Service rate – metrics 6
Figure 9: Routing table 7
Figure 10: Aggregate arrival rates 8

Introduction

The objective of this project is to model a Kubernetes-deployed microservice system and characterize the arrival and service processes. We chose to model Google's Microservices-Demo project, a small web application implementing an Online Boutique. The web application is composed of multiple microservices, implemented in various popular programming languages, see Figure 1.

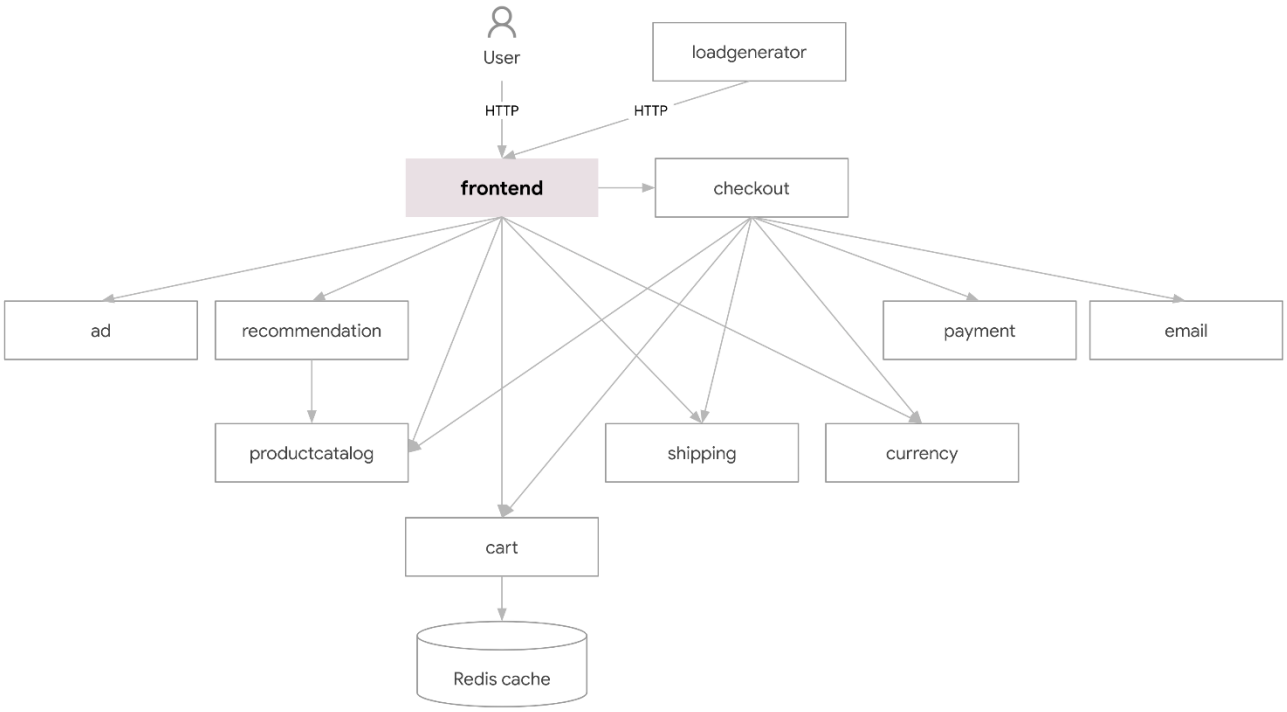

Figure 1: system architecture

We thus refined and implemented tracing and metric collection across all major services, collected data, modeled the system as a Jackson Network, and finally compared the queueing network predictions with actual measurements. The queueing network model we built accurately predicts steady-state arrival rates but makes minor errors in predicting steady-state queue length, and to a lesser extent, the response time.

Instrumentation and Execution

The Microservices-Demo application can be deployed in several ways. While it defaults to Google Cloud, it can also be deployed locally using Skaffold. This local setup requires a few key tools: Docker for container management, Kind to provision the Kubernetes cluster, and Skaffold to orchestrate the build and deployment process. Only deploying the project is not enough tho, we also need a way to monitor and collect data on the running services.

Fortunately, the project supports further customization through Kustomize extensions like the google-cloud-operations extension, which integrates tracing into specific services and exports data to Google Cloud. Unfortunately, tracing has not been implemented for all services in the source code, while metrics collection is totally absent.

For this reason, we had to modify the existing Opentelemetry code instrumentation and kustomize configuration to implement tracing and stats collection for all major services. In this adapted version, telemetry data is persisted into local JSON files for offline analysis rather than being uploaded to the cloud.

To run the simulation, use the run_simulation.sh bash script. The script will apply kustomization patches automatically, start the kind cluster, begin the Skaffold building process, and run the get_tp_mapping.sh script. This script produces two JSON files: pod_ip.json and service_cluster_ip.json, useful for later trace analysis. Opentelemetrycollector, the pod running trace and metric collection, will store the collected data in the otel-data directory.

Collected Metrics

For each service, we decided to collect three metrics:

1. Requests Count: from which we can calculate the arrival rate, dividing it by the time interval of the same arrivals. We need the arrivals to be exponentially distributed for later modeling, which can be configured in the load generator with Locust.
2. Request Duration: from which we would ideally obtain the response time of the service. Unfortunately, in our implementation, we also accounted for the duration of downstream services, making this metric measure the residence time instead of the response time, requiring some adjustment to the analysis strategy in subsequent phases.
3. Active Requests: measures the number of requests the service is currently processing. Not the most accurate measure of queue length in fast or downstream services, in which requests are often missed. However, in upstream services, it represents a reliable way to measure the average number of active requests (in the frontend, this would be equal to the number of requests in the whole system).

Traces and metrics processing

The collected metrics and traces should be transferred to the data directory or any directory in the Queueing-network-modeling-of-Online-Boutique project by setting the WKDIR environment variable. Then, by running the corresponding scripts (recalculate_traces.sh or recalculate_metrics.sh), the data will be processed, and the resulting JSON files will be generated accordingly. These files should be moved to the jupyter_data and appropriately named for the notebook to pick up and analyze. For more details on metric and trace processing, see the corresponding Python scripts.

Modeling and Analysis

The load generator can hit the frontend in different endpoints: “GET /”, “POST /setCurrency”, “GET /product/{id}”, “GET /cart”, “POST /cart”, “POST /cart/empty”, “POST /cart/checkout”, and “GET /logout”. These interactions are implemented in the locust file as eight different tasks, each hitting different endpoints with randomized requests and each generated with different probabilities. Tasks are generated according to an exponential distribution with a preconfigured rate of generation, maximum user number (the users are units of generation), and user spawn rate. Since the tasks are generated exponentially, we can expect a mostly exponential request arrival rate at the frontend (not a perfect exponential distribution because the checkout and addToCart tasks hit more endpoints sequentially). The following is a sequence of queueing networks modeling how every endpoint request is processed by the microservice system.

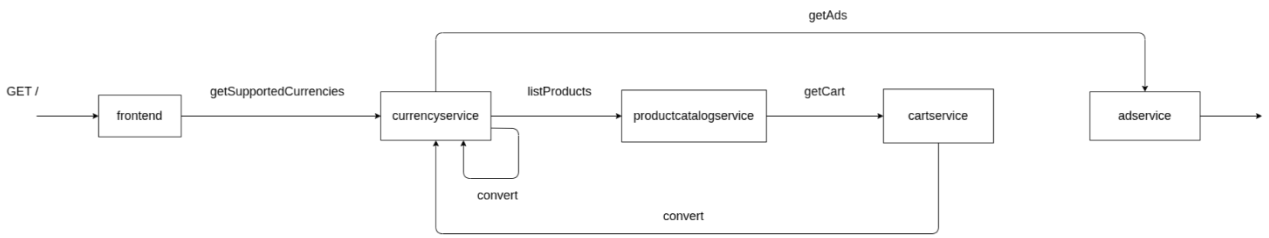

Figure 2: Home QN

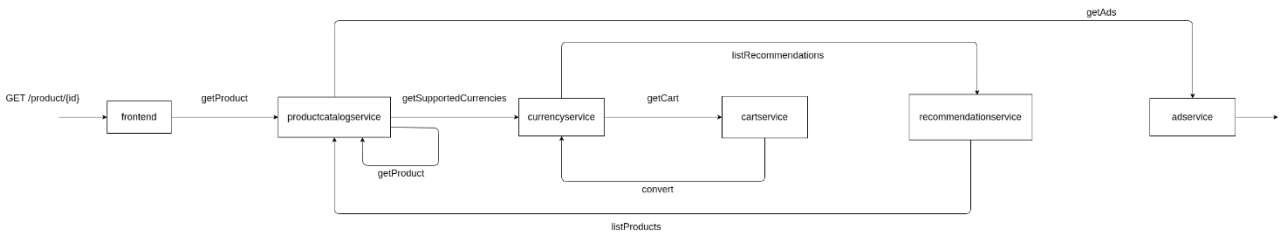

Figure 3: Product QN

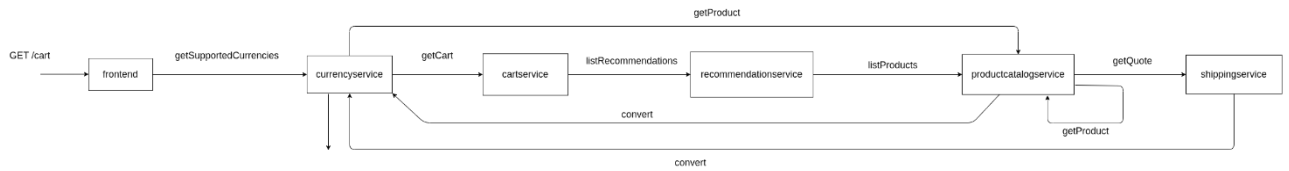

Figure 4: ViewCart QN

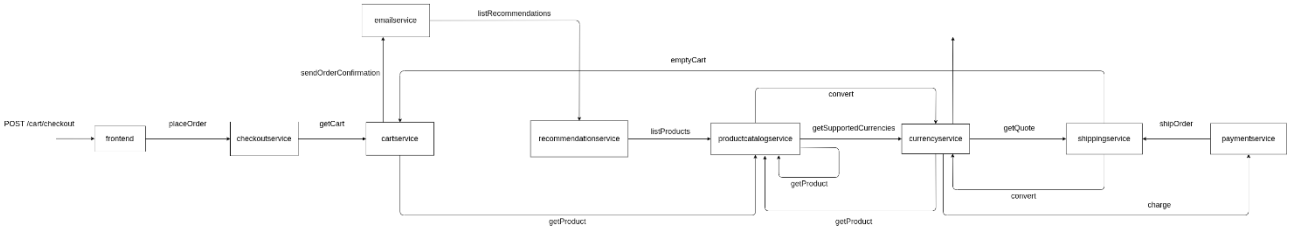

Figure 5: PlaceOrder QN

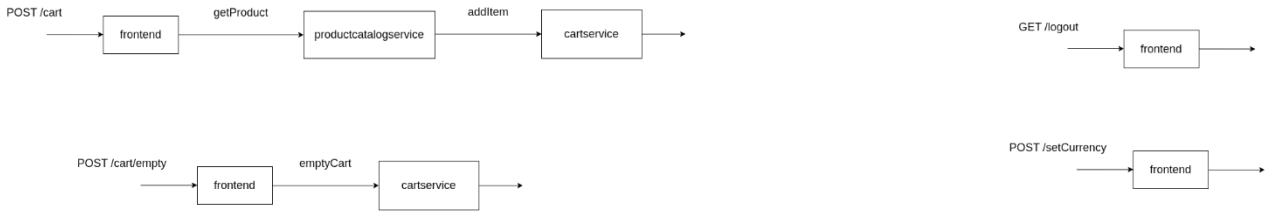

Figure 6: AddToCart QN, EmptyCart QN, Logout QN and SetCurrency QN

The queueing network for the entire Microservice-Demo system is constructed by taking the union of the individual endpoint networks. This unified graph incorporates all original nodes and arcs; overlapping arcs with identical sources and destinations are merged. The resulting network adheres to the following properties:

1. The network is open. In other words, it contains at least one node with exogenous arrivals (the frontend) and multiple nodes with exogenous departures (frontend, cartservice, currencyservice, and adservice).
2. Each node has an infinite queue (not true, but if we work at a low enough load such that requests are not lost midway, then this condition is irrelevant), and some number of exponential servers with a given service rate.
3. External arrivals are Poisson, given the fact that the load generator creates tasks exponentially.
4. Request routing is dependent on history. However, to simplify modeling, we assume routing independence and use statistical probabilities to model node transitions.

These assumptions allow the system to be solved as a Jackson Network. The whole implementation and analysis process can be reproduced using the dedicated Jupyter notebook “analytic_model.ipynb” in the corresponding repository of this project.

Determining service rates

Before building the Jackson network model, we will need to determine the service rates of each service in the system. To do so, we will exploit the data we collected in a low-load simulation run of the system. In such runs, we can safely approximate the service time with the response time since the waiting time before service will be negligible:

$$ResponseTime = WaitTime + ServiceTime$$

$$WaitTime \approx 0 \Rightarrow ResponseTime \approx ServiceTime$$

This reasoning works out perfectly in leaf services, where there is no downstream service, so the response time is not polluted. For upstream services, things become quite complicated. We thought two possible solutions to this problem:

1. We drop the requests_duration metric and instead use traces to measure response time. This can be done by taking the difference between the start time of span for a incoming call and the start time of the next outgoing span (or end time of the same span if it is the last in the trace).
2. We retain the requests_duration metric and adjust the response time of upstream services by subtracting the contributions of downstream services. These contributions are calculated by multiplying the average number of calls for that downstream service by the average response time.

We obtain in this way a reasonable estimate of service time for all pods in the system and can thus compute service rates assuming exponential service. The results are shown in the following figures, Figure 7 and Figure 8:

```
Service rates for each service computed with collected traces:
frontend - 1081.9230000716905
adservice - 466.64630432650256
recommendationservice - 448.79627096852084
cartservice - 352.7325352813
checkoutservice - 1546.0611637898162
paymentservice - 359.1265507073078
shippingservice - 759.2274671521805
emailservice - 228.13354183322042
currencyservice - 173.3263063797736
productcatalogservice - 630.0628756438136
```

Figure 7: Service rate - traces

```
Service rates for each service computed with collected metrics:
frontend - 31.330696985264034
adservice - 3954.8022598870025
recommendationservice - 284.7289874633431
cartservice - 1096.1763411234535
checkoutservice - 52.953086851664615
paymentservice - 3285.714285714285
shippingservice - 11954.708939153048
emailservice - 3384.3227541689084
currencyservice - 4613.916500994038
productcatalogservice - 15342.389173555272
```

Figure 8: Service rate – metrics

From the two figures, we can observe that the service rates computed from traces are more balanced, while for the version computed with collected residence time, we see important differences in service rates between upstream and leaf services. This may be due to the fact that in the metric method, upstream residence time also accounts for all types of delays

requests may encounter during the routing process, while for the trace method, these delays are theoretically distributed.

Building the routing table

The routing table will act as the specification of the entire analytical model. Once built, we will just need to change the exogenous arrival vector to adapt the model to predict performance metrics of the same system under different loads. To build the routing table, we will make use of the trace data again. This data should be collected, preferably, in long periods of time, and contain as many traces as possible to accurately model routing probabilities.

The collected traces are filtered to discard malformed or incomplete ones, with the spans of the remaining ones ordered by the time of creation. The span order will tell us the order in which services were called for that request. This information, averaged out for many traces, gives us the routing probabilities between services. Finally, the first and the last span of the trace are also useful for network topology confirmation and computing system departure probabilities in each pod.

```
Full routing matrix:
frontend - 0
adservice - 1
recommendationservice - 2
cartservice - 4
checkoutservice - 8
paymentservice - 7
shippingservice - 5
emailservice - 9
currencyservice - 6
productcatalogservice - 3

        0       1       2       3       4       5       6       7       8       9
0       0.000   0.000   0.000   0.502   0.032   0.000   0.342   0.000   0.031   0.000
1       0.000   0.000   0.000   0.000   0.000   0.000   0.000   0.000   0.000   0.000
2       0.000   0.000   0.000   1.000   0.000   0.000   0.000   0.000   0.000   0.000
3       0.000   0.088   0.000   0.671   0.054   0.040   0.147   0.000   0.000   0.000
4       0.000   0.000   0.198   0.033   0.000   0.000   0.602   0.000   0.000   0.033
5       0.000   0.000   0.000   0.000   0.125   0.000   0.875   0.000   0.000   0.000
6       0.000   0.051   0.133   0.122   0.194   0.010   0.409   0.010   0.000   0.000
7       0.000   0.000   0.000   0.000   0.000   1.000   0.000   0.000   0.000   0.000
8       0.000   0.000   0.000   0.000   1.000   0.000   0.000   0.000   0.000   0.000
9       0.000   0.000   1.000   0.000   0.000   0.000   0.000   0.000   0.000   0.000
```

Figure 9: Routing table

The routing table confirms the queueing network architecture we built earlier by statically examining the source code of the system.

Solving Traffic Equations

Having obtained the routing matrix and the exogenous arrival rate vector (which can be built by simply inserting the desired arrival rate at the frontend in position 0 of a vector of dimension 10×1 full of zeros), we can solve the corresponding traffic equation for the Jackson network we have built and obtain the aggregate arrival rates at each service node. The following equation was solved with the help of numpy's linalg.solve() function:

$$(I - Q^T)\vec{\lambda} = \vec{\gamma}$$

Where I is the identity matrix, Q is the routing matrix, γ is the exogenous arrival rate vector, and λ is the aggregated arrival rate vector. The results were the following:

```
Aggregate arrival rates for each service:
frontend - 24.0
adservice - 13.559524316951531
recommendationservice - 15.008567409743819
cartservice - 22.512595371041293
checkoutservice - 0.7406333915860364
paymentservice - 0.7406333915860361
shippingservice - 5.943480670048161
emailservice - 0.7406333915860361
currencyservice - 73.48484719321422
productcatalogservice - 111.74050551979877
```

Figure 10: Aggregate arrival rates

Comparison with empirical results

After having solved the traffic equations, we obtain the aggregate arrival rate vector, which enables us to compute performance metrics for each node in the network and for the whole network. We worked under two alternative assumptions about the behavior of each service node:

1. All service nodes behave as M/M/1 queues. The simplest assumption. It was formulated considering the limited resources at disposal during the local executions of the system.
2. Services nodes are assigned a service capacity according to the number of concurrent threads assigned to them, allowing us to model concurrency in some services.

To compute the performance metrics, we first calculated the average queue length at steady state for each node based on its type. We then obtained the average number of requests in the system by summing these values. This allowed us to apply Little's Law to determine both the average system response time at steady state and the average response time at each service node, using the system arrival rate (i.e., the exogenous arrival rate at the frontend) and the aggregate arrival rate at each node, respectively. The values we computed in this way are then compared with actual measured or estimated performance metrics under different combinations of assumptions (simple-queueing-model×service-from-traces, simple-queueing-model×service-from-metrics, concurrent-queueing-model×service-from-traces, and concurrent-queueing-model×service-from-metrics). The most accurate prediction was obtained with the all M/M/1 queue assumption using service rates obtained from collected residence time metrics.

Table 1: System-wide performance metrics (all M/M/1 queues)

	predicted - metrics	**predicted - traces**	**actual -metrics**
System L	3.39	1.12	3.5
System W	0.141	0.0467	0.150
System γ	24.0	24.0	24.0

Actual metrics in Table 1 are obtained from metrics collected directly from the frontend.

As for the performance predictions on single service nodes, we have:

Table 2: Service performance metrics

	predicted - metrics	predicted - traces	actual - metrics	actual - traces
Frontend L	3.27	0.0227	3.10	0.0451
Frontend W	0.136	0.000945	0.129	0.00188
Frontend λ	24.0	24.0	24.0	24.0
Adservice L	0.00344	0.0299	0.00236	0.0611
Adservice W	0.000254	0.00221	0.000184	0.00476
Adservice λ	13.6	13.6	12.8	12.8
Recommservice L	0.0556	0.0346	0.363	0.359
Recommservice W	0.00371	0.00231	0.0250	0.0247
Recommservice λ	15.0	15.0	14.5	14.5
Cartservice L	0.0210	0.0682	0.0290	0.380
Cartservice W	0.000931	0.00303	0.00130	0.0171
Cartservice λ	22.5	22.5	22.3	22.3
Checkoutservice L	0.0142	0.000479	0.0441	0.00199
Checkoutservice W	0.0192	0.000647	0.0567	0.00256
Checkoutservice λ	0.741	0.741	0.778	0.778
Paymentservice L	0.000225	0.00207	0.000178	0.00274
Paymentservice W	0.000304	0.00279	0.000242	0.00374
Paymentservice λ	0.741	0.741	0.733	0.733
Shippingservice L	0.000497	0.00789	0.00199	0.0413
Shippingservice W	8.37e-5	0.00133	0.000349	0.00726
Shippingservice λ	5.94	5.94	5.69	5.69
Emailservice L	0.000219	0.00326	0.000485	0.0228
Emailservice W	0.000296	0.00440	0.000624	0.0293
Emailservice λ	0.741	0.741	0.777	0.777
Currencyservice L	0.0162	0.736	0.0198	1.62
Currencyservice W	0.000220	0.0100	0.000283	0.0232
Currencyservice λ	73.5	73.5	69.9	69.9
Prodcatservice L	0.00734	0.216	0.0298	1.39
Prodcatservice W	6.57e-5	0.00193	0.000265	0.0123
Prodcatservice λ	112	112	112	112

In Table 2, we see the "actual – metrics" field that shows metrics computed by estimating response time with the METRICS method, and the "actual – traces" field that instead shows metrics computed by estimating response time with the TRACES method.

We can observe that the aggregate arrival rates obtained by solving the traffic equations are almost equal to the measured empirical arrival rates, confirming the quality of the trace data and the correctness of the network model. There is overwhelming agreement between the performance metrics computed using the service rate derived from the METRICS method and the empirical metrics obtained in the same way (assuming M/M/1 queues for all nodes), while

metrics computed with the TRACES method fail to do so. This could be caused by delays not being included in service rates computed from traces, or more likely, some undiscovered bug in trace processing.

As for the correctness of the model under different loads, we also experimented with lower loads and higher ones: lower loads presented problems in the stabilization of measured metrics, which often fluctuate given the lower number of requests in the system, making it difficult to reach a stable state in which the model could work. For higher arrival rates (bigger than 20 Hz), the system is CPU-capped and stops behaving like a Jackson Network, having consistently much longer response times than predicted and much lengthier queues.